

Universidad Nacional de Ingeniería
Escuela Profesional de Ingeniería de Software – Facultad de Ingeniería Industrial y de Sistemas
SW505 – Construcción de Software I

Lab Sem03-2 – Anticipación al Cambio – Estándares de Desarrollo

Objetivos del Laboratorio

- **Identificar puntos de variación** en un problema de negocio y encapsularlos con el mecanismo adecuado (polimorfismo, inyección de dependencias, configuración, Strategy).
- **Aplicar estándares de codificación** (nomenclatura, formato, comentarios, organización) en código propio y generado.
- **Refactorizar** código rígido hacia uno extensible, sin romper su comportamiento externo (Fowler, 2018).
- **Justificar decisiones de diseño** con criterios de costo de cambio y reutilización.

FASE 0 — Arranque

Pregunta detonadora al grupo:

"Si mañana el negocio pide aceptar criptomonedas, y el código de pagos actual tiene un if (tipo.equals("tarjeta")) ... else if (tipo.equals("yape")) ..., ¿cuántos archivos tendrían que tocar? ¿Cuántos tests se rompen?"

Se anotan respuestas en pizarra. Se introduce la idea central de la sesión:

"El cambio no se evita; se aísla." (SWEBOK v4, Construcción)

FASE 1 — El código que duele

Escenario: Sistema de procesamiento de pedidos de una tienda peruana. El siguiente código es **deliberadamente rígido**.

PedidoRigido.java

```
public class PedidoRigido {  
  
    public double calcularTotal(String tipoCliente, double subtotal, String metodoPago  
) {  
        double total = subtotal;  
  
        // Descuento según cliente
```

```

        if (tipoCliente.equals("regular")) {
            // sin descuento
        } else if (tipoCliente.equals("vip")) {
            total = total * 0.85;
        } else if (tipoCliente.equals("navidad")) {
            total = total * 0.80;
        }

        // Procesamiento según método de pago
        if (metodoPago.equals("tarjeta")) {
            System.out.println("Cobrando S/ " + total + " con tarjeta");
        } else if (metodoPago.equals("yape")) {
            System.out.println("Cobrando S/ " + total + " con Yape");
        } else if (metodoPago.equals("efectivo")) {
            System.out.println("Cobrando S/ " + total + " en efectivo");
        }

        return total;
    }
}

```

Preguntas guía al estudiante (en parejas):

1. ¿Cuántos **puntos de variación** hay? (McConnell, 2004 – Cap. 5)
2. ¿Qué pasa si el negocio agrega "descuento por Black Friday"?
3. ¿Qué pasa si se agrega "Plin" como método de pago?
4. ¿Qué pasa si el descuento VIP cambia de 15% a 20%?
5. ¿Qué pasa si el umbral de "cliente VIP" cambia según el monto?

"Un punto de variación es una decisión de diseño que se sabe que puede cambiar. Encapsularlo significa que el cambio afecta solo al código dentro de esa frontera."

FASE 2 — Refactor a Polimorfismo

Meta: Reemplazar el `if/else` de métodos de pago por polimorfismo.

Paso 2.1 — Definir la interfaz

```

public interface MetodoPago {
    void procesar(double monto);
    String nombre();
}

```

Paso 2.2 — Implementaciones

```

public class PagoTarjeta implements MetodoPago {
    @Override

```

```

    public void procesar(double monto) {
        System.out.println("Cobrando S/ " + monto + " con tarjeta");
    }
    @Override
    public String nombre() { return "Tarjeta"; }
}

public class PagoYape implements MetodoPago {
    @Override
    public void procesar(double monto) {
        System.out.println("Cobrando S/ " + monto + " con Yape");
    }
    @Override
    public String nombre() { return "Yape"; }
}

public class PagoEfectivo implements MetodoPago {
    @Override
    public void procesar(double monto) {
        System.out.println("Cobrando S/ " + monto + " en efectivo");
    }
    @Override
    public String nombre() { return "Efectivo"; }
}

```

Paso 2.3 — Uso

```

public void cobrar(MetodoPago metodo, double monto) {
    metodo.procesar(monto);
}

```

Ejercicio: Agregar `PagoPlin` sin tocar ninguna clase existente.

La idea:

"Agregar un nuevo método de pago costó 1 archivo nuevo, 0 archivos modificados. Ese es el ahorro del aislamiento."

Nota Python equivalente:

```

from abc import ABC, abstractmethod

class MetodoPago(ABC):
    @abstractmethod
    def procesar(self, monto: float) -> None: ...

class PagoYape(MetodoPago):
    def procesar(self, monto: float) -> None:
        print(f"Cobrando S/ {monto} con Yape")

```

FASE 3 — Strategy + Inyección de Dependencias

Meta: Separar los descuentos (algoritmos completos intercambiables) del Pedido (contexto).

Paso 3.1 — Interfaz Strategy

```
public interface EstrategiaDescuento {  
    double aplicar(double subtotal);  
    String descripcion();  
}
```

Paso 3.2 — Estrategias concretas

```
public class SinDescuento implements EstrategiaDescuento {  
    @Override public double aplicar(double subtotal) { return subtotal; }  
    @Override public String descripcion() { return "Sin descuento"; }  
}  
  
public class DescuentoVip implements EstrategiaDescuento {  
    @Override public double aplicar(double subtotal) { return subtotal * 0.85; }  
    @Override public String descripcion() { return "VIP 15%"; }  
}  
  
public class DescuentoNavidad implements EstrategiaDescuento {  
    @Override public double aplicar(double subtotal) { return subtotal * 0.80; }  
    @Override public String descripcion() { return "Navidad 20%"; }  
}
```

Paso 3.3 — Inyección de dependencias

```
public class Pedido {  
    private final EstrategiaDescuento estrategia;  
    private final MetodoPago metodoPago;  
  
    // DI por constructor: Los colaboradores Llegan desde fuera  
    public Pedido(EstrategiaDescuento estrategia, MetodoPago metodoPago) {  
        this.estrategia = estrategia;  
        this.metodoPago = metodoPago;  
    }  
  
    public double procesar(double subtotal) {  
        double total = estrategia.aplicar(subtotal);  
        metodoPago.procesar(total);  
        return total;  
    }  
}
```

Paso 3.4 — Composición en el `main`

```
public class Main {
    public static void main(String[] args) {
        Pedido pedido = new Pedido(
            new DescuentoVip(),
            new PagoYape()
        );
        pedido.procesar(200.0); // VIP 15% + Yape
    }
}
```

Preguntas guía:

- ¿Qué gana `Pedido` al no instanciar sus colaboradores?
- ¿Cómo se testea `Pedido` sin tocar base de datos o pasarela real? (Se menciona el doble de prueba / mock).
- ¿Qué diferencia hay entre polimorfismo simple y Strategy? (El PDF: "aquí el objeto completo representa un algoritmo".)

Ejercicio: Crear `DescuentoBlackFriday` con 30% y usarlo sin modificar `Pedido` ni `Main` más allá de una línea.

FASE 4 — Parámetros de configuración

Meta: Externalizar valores que cambian por entorno.

Paso 4.1 — Antes (mal)

```
public class ServicioPago {
    private static final String URL_PASARELA = "https://sandbox-pagos.empresa.pe/api";
    private static final double UMBRAL_VIP = 500.0;
    // ...
}
```

Paso 4.2 — Después (bien)

`src/main/resources/application.properties:`

```
properties
pagos.url=https://sandbox-pagos.empresa.pe/api
pagos.umbra1.vip=500.0
pagos.timeout.segundos=30
```

Lectura con `java.util.Properties` (sin frameworks):

```

import java.io.InputStream;
import java.util.Properties;

public class Configuracion {
    private final Properties props = new Properties();

    public Configuracion() throws Exception {
        try (InputStream in = getClass()
            .getClassLoader()
            .getResourceAsStream("application.properties")) {
            props.load(in);
        }
    }

    public String getUrlPagos() { return props.getProperty("pagos.url"); }
    public double getUmbralVip() { return Double.parseDouble(props.getProperty("pagos.
umbral.vip")); }
}

```

Preguntas guía:

- ¿Qué pasa si el equipo de QA necesita probar con otro umbral sin recompilar?
- ¿Qué riesgo hay al hardcodear credenciales? (*Semana 11 – seguridad*).

Nota Python equivalente: `os.getenv("PAGOS_URL") + python-dotenv`.

FASE 5 — Estándares de codificación

Meta: Aplicar los 4 ejes del PDF (nomenclatura, formato, comentarios, organización) con herramientas.

Paso 5.1 — Checklist de nomenclatura (McConnell, 2004 – Cap. 11)

Regla	Mal	Bien
Nombres revelan intención	d, x, tmp	subtotal, descuentoAplicado
Clases = sustantivos	Procesar	ProcesadorPago
Métodos = verbos	pago()	procesarPago()
Sin abreviaturas crípticas	calcTotCliVip()	calcularTotalClienteVip()
Buscables	0.85	FACTOR_DESCUENTO_VIP = 0.85

Paso 5.2 — Comentarios: el porqué, no el qué

Evitar:

```
// incrementa i en 1
i++;
```

Preferir:

```
// La pasarela pagina de 50 en 50; avanzamos al siguiente lote
i++;
```

Mini-ejercicio: Ejecutar el formateador sobre el código del laboratorio y observar el diff. Discutir: "¿Por qué delegar el formato libera energía para discutir diseño?"

FASE 6 — Reto integrador en parejas

Consigna: Extender el sistema con **dos nuevos puntos de variación**, aplicando el mecanismo adecuado y respetando los estándares.

Escenario extendido

La tienda ahora requiere:

1. **Notificaciones** al confirmar el pedido: email, SMS o WhatsApp.
2. **Cálculo de impuestos** según región: Lima (18% IGV), Arequipa (18%), Selva (10% promocional).

Requisitos del entregable

- Identificar cada punto de variación con un comentario `// PUNTO DE VARIACIÓN:` y justificar el mecanismo elegido.
- Aplicar mínimo **dos mecanismos distintos** (Strategy, polimorfismo, DI, configuración).
- **Cero** `if/else` o `switch` sobre strings para seleccionar comportamiento.
- Nomenclatura consistente (verbos en métodos, sustantivos en clases).
- Al menos un valor externalizado a `application.properties`.
- Un archivo `README.md` de 10 líneas explicando la estructura de paquetes.

Estructura sugerida

text

```

src/
├─ main/
│   └─ java/
│       └─ pe/empresa/pedidos/
│           ├── Main.java
│           ├── domain/
│           │   ├── Pedido.java
│           │   └─ EstrategiaDescuento.java
│           ├── pago/
│           │   ├── MetodoPago.java
│           │   └─ PagoYape.java
│           ├── notificacion/
│           │   ├── Notificador.java
│           │   └─ NotificadorEmail.java
│           └─ impuesto/
│               ├── CalculadoraImpuesto.java
│               └─ ImpuestoLima.java
└─ resources/
    └─ application.properties

```

Cita de cierre (McConnell, 2004)

"La buena construcción no es escribir código rápido; es escribir código que sobreviva al cambio."

Anexos para el estudiante

A. Equivalencias Java ↔ Python de los mecanismos

Mecanismo	Java	Python
Polimorfismo	interface + implements	ABC + @abstractmethod
DI	Constructor / @Inject	Argumento de <code>__init__</code>
Configuración	application.properties	.env + os.getenv
Strategy	interface + clases concretas	Igual, con clases o funciones de primera clase

B. Lecturas recomendadas

- McConnell, S. (2004). *Code Complete* — Caps. 5 (Construcción), 7 (Rutinas), 11 (Nombres).
- Gamma et al. (1994). *Design Patterns* — Cap. Strategy.
- Fowler, M. (2018). *Refactoring* — Cap. 3 (Bad Smells: Long Method, Switch Statements).
- SWEBOK v4 — Sección "Software Construction".

C. Comandos útiles

```
bash
```

```
# Compilar y ejecutar
```

```
javac -d out $(find src -name "*.java")
```

```
java -cp out pe.empresa.pedidos.Main
```

```
# Formatear con google-java-format
```

```
java -jar google-java-format.jar --replace src/**/*.java
```

```
# Checkstyle
```

```
java -jar checkstyle.jar -c google_checks.xml src/
```